

DyCODER: DYNAMICALLY INTERLEAVING CONTINUOUS AND DISCRETE REASONING

Mete Ismayilzada, Angelina Frolova, Carolyn Crichton

EPFL

{mete.ismayilzada, angelina.frolova, carolyn.crichton}@epfl.ch

ABSTRACT

Recent work has shown that augmenting large language models with latent-space reasoning can improve performance over purely textual chain-of-thought methods, but existing approaches rely on fixed, externally specified schedules for continuous reasoning. We introduce DyCODER, a framework for dynamic interleaving of continuous and discrete reasoning that enables models to autonomously decide when to enter and exit latent reasoning, as well as how long to reason in continuous space. By learning variable-length, model-controlled latent spans, DyCODER removes rigid phase boundaries and adapts reasoning depth to problem complexity. Experiments on mathematical and logical reasoning tasks with GPT-2 and Qwen2.5-0.5B demonstrate that DyCODER matches or outperforms prior latent-reasoning methods while improving efficiency.

1 INTRODUCTION

Large language models (LLMs) have recently shown strong performance on complex reasoning tasks, particularly when augmented with explicit reasoning strategies such as chain-of-thought (CoT) prompting (Wei et al., 2022). However, CoT relies entirely on discrete natural-language tokens, which may not fully exploit the representational capacity of a model’s internal hidden states (Turpin et al., 2023; Paul et al., 2024). Motivated by this limitation, a growing body of work has explored latent-space reasoning, treating hidden states as a continuous representation of intermediate reasoning steps, often referred to as continuous thoughts. Across a range of benchmarks, these approaches have demonstrated substantial improvements over standard CoT methods.

A foundational idea in this line of work is the interleaving of continuous and discrete reasoning via latent representations. COCONUT explores this idea by introducing latent rollouts, in which a model’s hidden states are recursively fed back as embeddings, allowing reasoning to unfold in continuous space before returning to natural-language output (Hao et al., 2024). While effective, COCONUT enforces a rigid two-phase structure consisting of a fixed-length block of latent reasoning followed by natural-language generation. The model cannot determine when to transition between modalities or how many latent steps are required, and prior work notes that excessively long latent blocks can destabilize reasoning or degrade accuracy.

HybridCoT (Shen et al.), addresses some of this rigidity by interleaving latent and textual reasoning steps. Although this improves stability, the interleaving schedule remains externally specified, with each latent segment containing a predetermined number of continuous updates. As a result, the model still lacks the ability to adapt the length or placement of latent reasoning based on task difficulty. In both COCONUT and HybridCoT, the duration of latent reasoning is fixed a priori rather than learned, limiting flexibility and potentially leading to inefficient reasoning trajectories.

To this end, we propose a new approach that enables Dynamic Interleaving of Continuous and Discrete Reasoning (DyCODER). Unlike prior approaches, our method does not rely on fixed schedules or external controllers. Instead, we propose an adaptive latent-space reasoning mechanism that allows the model to autonomously decide when to enter and exit latent reasoning. The model learns to regulate both the placement and length of latent reasoning spans as a function of problem complexity, yielding a variable-length, model-controlled alternation between continuous and discrete reasoning. This design more closely reflects the flexible interplay between internal reasoning and external articulation.

Our experiments with GPT-2 and Qwen2.5-0.5B models trained on math and logical reasoning datasets show that DyCODER outperforms or is on par with other approaches while being more efficient. We make our code available here ¹.

2 RELATED WORK

Chain of Thought (CoT) reasoning helps models solve multi-step problems by prompting them to write out intermediate steps in natural language before giving a final answer (Wei et al., 2022). Earlier work showed that writing steps provides a kind of scratch space that can make reasoning more reliable (Nye et al., 2021). Later studies found that models can produce these steps even without examples, using small cues, and that breaking problems into ordered subproblems can improve performance on harder tasks (Kojima et al., 2022; Zhou et al., 2022). Because the steps are written out, researchers can influence how much effort the model uses at test time and can inspect how it reaches an answer. Accuracy often improves when you sample several different step-by-step attempts and combine their final answers, which suggests that trying multiple paths can matter more than any single path (Wang et al., 2022). Trees of Thought (ToT) builds on this idea by treating each intermediate step as a branching point, so the model can explore different paths (Yao et al., 2023). However, other work shows these written steps can sound convincing while not reflecting what the model actually did internally, and newer studies test this by checking whether the steps truly affect the final answer, finding that plausible steps are not always true explanations (Turpin et al., 2023; Paul et al., 2024).

It is, however, unclear whether longer CoT helps because the words add useful reasoning or because generating more tokens gives the model more time to compute. Pause token training tests this by adding learned pause tokens that delay the final answer and allow extra internal updates before choosing an answer token (Goyal et al., 2023). Filler token studies show that even repeated meaningless tokens can sometimes help by adding computation steps, though learning to use them is hard and depends on the task and supervision (Pfau et al., 2024). Together, these results suggest intermediate token positions can act as extra compute even without meaningful reasoning content.

To address both cost and faithfulness, some methods try to keep the benefits of CoT without requiring the model to print long explanations. One approach, implicit CoT, uses knowledge distillation so a student model learns from a teacher model’s internal hidden state paths that come from explicit CoT, letting the student model answer faster without outputting the full trace (Deng et al., 2023). Another approach, hidden CoT decoding, packs the reasoning into short special tokens that guide the final generation, reducing token use while keeping performance similar to explicit CoT in several settings (Liu et al., 2024b). Overall, these methods use CoT mainly as a training aid, suggesting that intermediate reasoning can be stored and used more efficiently than natural language.

As methods moved from discrete reasoning (CoT) to continuous space reasoning, SoftCot was introduced to address the challenge of catastrophic forgetting that arose during the fine-tuning of models (Xu et al., 2025). SoftCot keeps the main model fixed and trains a small helper model to produce task-specific soft thought vectors that are mapped into the main model’s embedding space. This makes training cheaper and aims to preserve the main model’s general abilities. Overall, SoftCot adds continuous reasoning signals as extra input, without retraining the whole model to learn a new latent reasoning process. Recent work has mixed continuous and discrete reasoning by letting the model “think” in latent space, then return to normal text output. COCONUT explores this with latent rollouts, where the model feeds its own hidden states back as the next input embeddings (Hao et al., 2024). This lets it run several reasoning updates in continuous space before producing natural language. However, COCONUT uses a rigid two-phase setup: a fixed-length block of latent reasoning followed by text generation. The model does not choose when to switch modes or how many latent steps it needs, and overly long latent blocks can become unstable and hurt accuracy.

HybridCoT reduces some of this rigidity by interleaving latent and text steps, which can improve stability and preserve important symbolic details, especially in math (Shen et al.). But the interleaving schedule is still set from the outside, and each latent segment has a preset number of continuous updates. So in both COCONUT and HybridCoT, the amount and placement of latent reasoning is

¹<https://github.com/mismayil/dycoder>

Table 1: Statistics of the preprocessed MATH dataset.

Split	Size	Avg. # steps	Avg. # step difficulty
Train	11,330	4.45	2.24
Validation	119	4.34	2.20
Test	478	4.38	2.21

t , i.e. $h_t = H_t[t, :]$; $e(\cdot)$ is the token embedding function; W is the parameter of the language model head.

3.1 METHOD OVERVIEW

Similar to the COCONUT method, our method also switches LLM between a “language mode” and “latent mode”. In language mode, the model operates as a standard language model, autoregressively generating the next token. In latent mode, it directly utilizes the last hidden state as the next input embedding. This last hidden state represents the current reasoning state, termed as a “continuous thought”.

We use the same special tokens $\langle | \text{start-latent} | \rangle$ and $\langle | \text{end-latent} | \rangle$ to mark the beginning and end of the latent thought mode, respectively. For example, if the latent reasoning occurs between positions i and j , i.e. $x_i = \langle | \text{start-latent} | \rangle$ and $x_j = \langle | \text{end-latent} | \rangle$. When the model is in the latent mode ($i < t < j$), we use the last hidden state from the previous token to replace the input embedding, i.e. $E_t = [e(x_1), \dots, e(x_i), h_i, h_{i+1}, \dots, h_{t-1}]$. After the latent mode finishes ($t \geq j$), the input reverts to using the token embedding i.e. $E_t = [e(x_1), \dots, e(x_i), h_i, h_{i+1}, \dots, h_{t-1}, e(x_j), \dots, e(x_t)]$.

However, different from the COCONUT method, we do not enforce a rigid two-phase structure consisting of a fixed-length block of latent reasoning followed by natural language generation. Instead, we add a latent reasoning block after each natural language reasoning step to allow the model to “think” for each step separately. Additionally, we make the length of each block adaptive to the difficulty of the preceding natural language block. To achieve this, we annotate the steps of the training dataset with “difficulty” values and introduce modifications to the original loss function used in Hao et al. (2024), which we describe below.

3.2 TRAINING PROCESS

Similar to COCONUT method, we focus on a problem-solving setting where the model receives a question as input and is expected to generate an answer through a reasoning process. We also leverage language CoT data to supervise continuous thought by implementing a multi-stage training curriculum inspired by Deng et al. (2024). As shown in Figure 1, in the initial stage, the model is trained on regular CoT instances. In the subsequent stages, at the k -th stage, each natural language step is followed by $c \cdot \min(k, d)$ latent thoughts, where c is a hyperparameter controlling the number of latent thoughts multiplier and d is the difficulty level of the previous step. This setup ensures that the model adapts to the difficulty level of the steps gradually, with fixed latent thoughts per step in the beginning and a varying number of latent thoughts per step in the end.

In order to achieve this adaptive reasoning capability, we make a few modifications to the training procedure of COCONUT. Similar to the COCONUT method, during the training process, we optimize the normal negative log-likelihood loss for language tokens, and mask the loss on questions and latent thoughts. However, since in our setup, the model is responsible in deciding when to start and end latent reasoning mode, we do not mask the $\langle | \text{start-latent} | \rangle$ and $\langle | \text{end-latent} | \rangle$ tokens. While this minimal change allows the model to see the special tokens and where they are placed, the model still does not know how many latent thoughts to use since those tokens between the start and end latent tokens are masked out. This inevitably results in the model outputting the $\langle | \text{end-latent} | \rangle$ right after the $\langle | \text{start-latent} | \rangle$ token (sometimes after one latent token), which is what we also found in our initial experiments. To enable a true adaptive mechanism, we need to teach the model to only output the end latent token after a certain number of latent

tokens. While there could be different ways to achieve this, in our work, we opted for a simple method of adding a new loss objective that discourages the emission of the end latent token in latent mode. In particular, at each latent token generation, we accumulate the average probability of the `<|end-latent|>` token and add this sum (scaled by a weighting factor λ) to the language modeling loss. More formally, our final loss function is as follows:

$$\mathcal{L} = \mathcal{L}_{LM} + \frac{\lambda}{N_{latents}} * \sum_t p(<|end-latent|> | x_{\leq t}) * \mathbf{1}_{t \in latent}$$

3.3 INFERENCE PROCESS

The inference process for DyCODER is analogous to COCONUT where, in addition to standard language model decoding, we directly feed the last hidden state as the next input embedding in latent mode. However, instead of padding latent thoughts to a constant length, we let the model decide and switch to the latent mode only when the model generates the `<|start-latent|>` token and switch back to the language decoding when the model generates the `<|end-latent|>` token.

4 EXPERIMENTS

In this section, we outline the setup and the results for our experiments, providing a proof of concept for interleaving discrete and continuous reasoning. All training runs have been logged to WandB and can be found here ².

4.1 DATASETS

In this work, we focus on two math reasoning datasets, GSM8k Cobbe et al. (2021), and MATH Hendrycks et al. (2021), and one logical reasoning dataset ProsQA Hao et al. (2024). We use the versions of GSM8k and ProsQA from Hao et al. (2024), which have already been preprocessed to contain the CoT steps and extracted answers.

Since CoT steps in these datasets are relatively easy and uniform in their difficulty, we set the difficulty of each step to 1. MATH dataset, on the other hand, is a more challenging dataset containing real-world math problems that involve a heavy use of LaTeX commands. The original dataset from Hendrycks et al. (2021) only contains the problems and the solutions. We first preprocess it to break each solution into several sentences and treat each sentence as an individual step. Next, we filter problems with too many steps (more than 15) and assign a difficulty level to each step using the DeepSeek model (Liu et al., 2024a) and extract the final answer using regex. The data processing pipeline for the MATH dataset can be found in Figure 2.

²<https://wandb.ai/mismayil/dycoder/workspace>

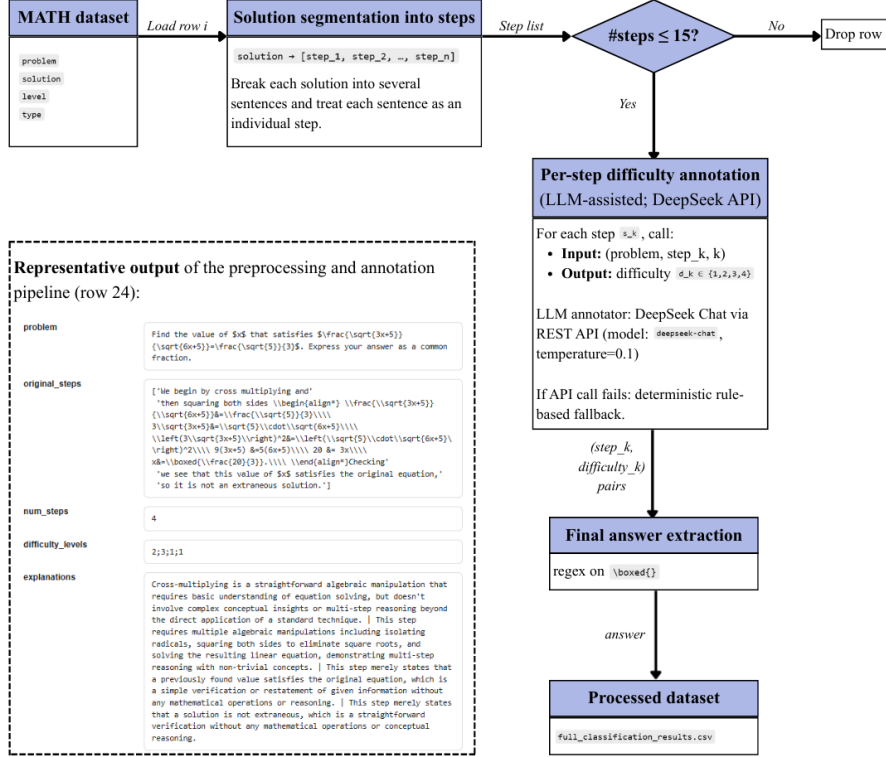


Figure 2: MATH dataset processing pipeline.

Final dataset statistics can be found in Table 1. The distribution of the steps and the difficulty levels can be found in Figure 3.

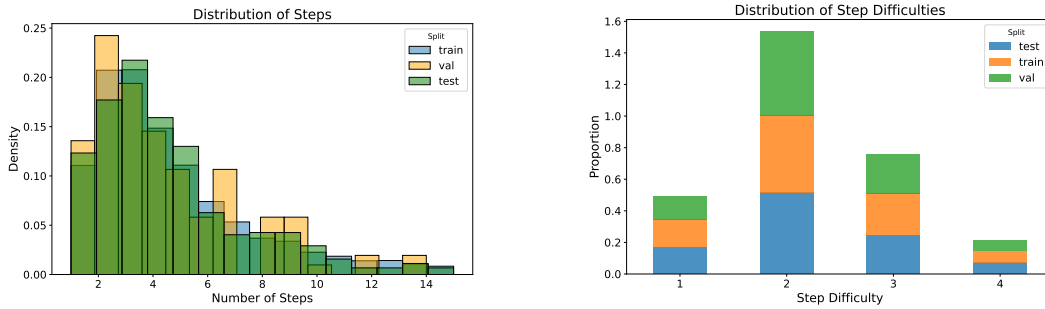


Figure 3: MATH data step and step difficulty distributions.

4.2 SETUP

We use a pre-trained GPT-2 model (124M) Radford et al. (2019) as the base model for GSM8k and ProsQA experiments following Hao et al. (2024). For the MATH dataset, we use a slightly bigger model Qwen2.5-0.5B³ (Qwen et al., 2024) since this is a harder dataset. Due to computational and time constraints, we limit our experiments to only these small models and datasets and train for 5

³We also experimented with SmoLLM-2 135M, however, the model struggled to learn the task.

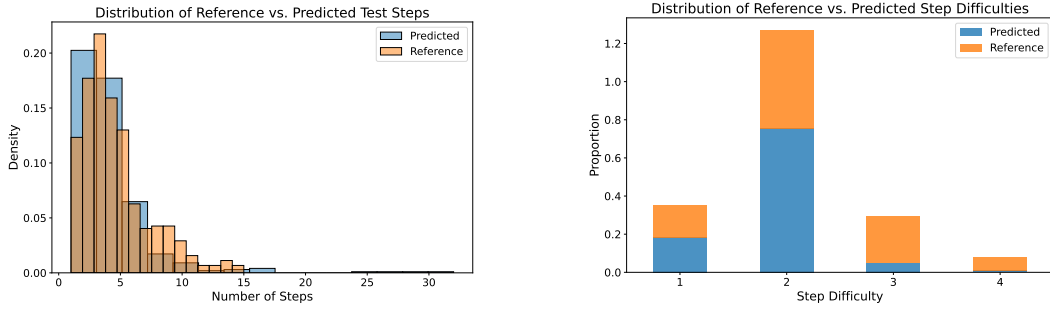


Figure 4: Comparing MATH data reference and predicted step and step difficulty distributions on the test set.

epochs. For all experiments, the learning rate is set to 1×10^{-4} with a weight decay of 0.01. As baselines, we only consider the vanilla CoT and COCONUT models as our approach is directly comparable to these (we also report other baseline performance values from Hao et al. (2024) where applicable). We also could not reproduce the HybridCoT paper as it is still under review with no public code provided and involves extra steps such as compressing text with sparse attention and a gisting method where preceding text tokens are selectively removed. For evaluation, we use the test split of each dataset and report the answer Accuracy (%) metric. To train the COCONUT model, we follow the original setup from Hao et al. (2024) and initialize the COCONUT model to the finetuned CoT model (trained for 5 epochs). For our DyCODER model, we train it from scratch from the pre-trained base model for 5 epochs, where the first epoch corresponds to training a CoT model (only 1 epoch). For all DyCODER trainings, we set the continuous thought multiplier $c = 1$ and latent persistence loss weight $\lambda = 0.1$.

4.3 RESULTS

Table 2 summarizes our main results across datasets and methods. We observe that our method outperforms all other methods except CoT on the GSM8k dataset with significantly fewer training epochs ($5\times$ more efficient). On the ProsQA dataset, our method achieves a comparable performance with $10\times$ fewer epochs. On the MATH dataset, our method outperforms both CoT and COCONUT methods and interestingly, COCONUT performs worse than CoT even though it is initialized to the CoT model when training. In Figure 4, we visualize and compare our model’s predicted step and step difficulty distribution to the respective reference distributions on the test set. We can see that the distributions are fairly close and the model has learned to “think” for a variable number of latent steps at each step.

Method	GSM8k		ProsQA		MATH	
	Acc. (%)	# Epochs	Acc. (%)	# Epochs	Acc. (%)	# Epochs
CoT	42.9*	25	77.5*	50	9.0	5
No-CoT	16.5*	25	76.7*	50	-	5
iCoT	30.0*	25	98.2*	50	-	5
Pause Token	16.4*	25	75.9*	50	-	5
COCONUT	34.1*	25	97.0*	50	5.1	5 (+5)
DyCODER (Ours)	38.4	5	67.0	5	9.4	5

Table 2: Performance comparison across GSM8k, ProsQA, and MATH. Results for the first two datasets are based on GPT-2 model, while results for MATH are based on Qwen2.5-0.5B model. * The results are from Hao et al. (2024).

CONCLUSION AND FUTURE WORK

In this work, we introduced DyCODER, a dynamic framework for interleaving continuous and discrete reasoning in large language models. Unlike prior latent-reasoning approaches that rely on fixed schedules or externally defined transitions, DyCODER enables models to autonomously regulate both the placement and duration of latent reasoning as a function of task complexity. This adaptive control yields flexible, variable-length reasoning trajectories that more closely mirror the interplay between internal computation and external language.

Due to computational and time constraints, our empirical evaluation focused on relatively small models, short training runs, limited datasets, and a small number of baselines. Despite these constraints, results on mathematical and logical reasoning tasks with GPT-2 and Qwen2.5-0.5B demonstrate that DyCODER achieves performance competitive with or superior to existing approaches while using latent computation more efficiently. These findings suggest that learned control over latent reasoning spans is a promising direction, even under restricted experimental settings.

Several avenues for future work remain. An important next step is scaling DyCODER to larger models and longer training regimes, as well as conducting more extensive comparisons with alternative methods such as HybridCoT and SoftCoT. Further investigation is also needed into different mechanisms for teaching models how many latent reasoning steps to use at each point, potentially through improved supervision or auxiliary objectives. In addition, analyzing the robustness of DyCODER under distributional shifts—such as variations in the number of reasoning steps or step difficulty—would provide deeper insight into its generalization properties. Finally, evaluating the approach on harder and more compositional benchmarks, including ProofWriter (Tafford et al., 2021) and Granular (Gao et al., 2025), would help assess its effectiveness on complex, multi-step reasoning tasks. Finally, future work could explore progressively removing textual reasoning tokens in later stages of training to further improve token efficiency while preserving reasoning performance.

REFERENCES

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Yuntian Deng, Kiran Prasad, Roland Fernandez, Paul Smolensky, Vishrav Chaudhary, and Stuart Shieber. Implicit chain of thought reasoning via knowledge distillation. *arXiv preprint arXiv:2311.01460*, 2023.
- Yuntian Deng, Yejin Choi, and Stuart Shieber. From explicit cot to implicit cot: Learning to internalize cot step by step. *arXiv preprint arXiv:2405.14838*, 2024.
- Silin Gao, Antoine Bosselut, Samy Bengio, and Emmanuel Abbe. Abstral: Augmenting llms’ reasoning by reinforcing abstract thinking. *arXiv preprint arXiv:2506.07751*, 2025.
- Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. *arXiv preprint arXiv:2310.02226*, 2023.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.

- Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. Can language models learn to skip steps? *Advances in Neural Information Processing Systems*, 37:45359–45385, 2024b.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show your work: Scratchpads for intermediate computation with language models. 2021.
- Debjit Paul, Robert West, Antoine Bosselut, and Boi Faltings. Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning. *arXiv preprint arXiv:2402.13950*, 2024.
- Jacob Pfau, William Merrill, and Samuel R Bowman. Let’s think dot by dot: Hidden computation in transformer language models. *arXiv preprint arXiv:2404.15758*, 2024.
- A Yang Qwen, Baosong Yang, B Zhang, B Hui, B Zheng, B Yu, Chengpeng Li, D Liu, F Huang, H Wei, et al. Qwen2. 5 technical report. *arXiv preprint*, 2024.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Shannon Zejiang Shen, Rulin Shao, Chenyu Wang, Songlin Yang, Vincent-Pierre Berges, Gargi Ghosh, Pang Wei Koh, Luke Zettlemoyer, Yoon Kim, Jason E Weston, et al. Hybridcot: Interleaving latent and text chain-of-thought for efficient reasoning. In *NeurIPS 2025 Workshop on Efficient Reasoning*.
- Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. Proofwriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 3621–3634, 2021.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965, 2023.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot: Soft chain-of-thought for efficient reasoning with llms. *arXiv preprint arXiv:2502.12134*, 2025.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.